React + Django: Task Manager

Full-stack CRUD · Django REST Framework · MySQL · Vite · Tailwind



Md Tausif Hossain

Your Full-stack Instructor

Tech Lead @ DevTechGuru · Founder, **TechnicalBind**

9+ years shipping production software

Instructor @ **Ostad**: ostad.app/instructor/73688

tausif.bd · GitHub · LinkedIn

What we'll cover

Project **goal**, the feature list and the REST contract

Backend setup: virtualenv, Django project + app, `.env`, MySQL

The API: model → migration → serializer → viewset → router

Frontend setup: Vite, Tailwind v4, folder structure

A **service module** that owns every `fetch` call

State design in `App.jsx` + the three presentational components

Running both servers, the errors you will hit, then bonus work

The Project

What we're building

Task Manager — add a task, tick it off, delete it. React on the front, Django REST Framework and MySQL on the back.

- 📝 **Create** a task: title + optional description
- ✅ **Toggle** complete / incomplete, with a live `3 of 7 complete` counter
- 🗑️ **Delete** a task
- 🔌 **REST API** over MySQL — six endpoints from one viewset
- ⚡ **No page reloads** — the list updates from React state
- 🧭 One `services/` module owns every HTTP call

The two halves

```
Browser
  React (Vite dev server, :5173)
    components/  →  App.jsx  →  services/taskApi.js
                                        │  fetch, JSON over HTTP
                                        ▼
  Django (runserver, :8000)
    urls.py → views.py → serializers.py → models.py
                                        │  Django ORM
                                        ▼
  MySQL   task_manager_db . tasks_task
```

Two servers run at once. The browser talks to Django over HTTP; Django talks to MySQL through the ORM. Neither side imports the other.

How one action flows

```
User submits TaskForm
  → onAddTask({ title, description, is_completed: false })
    → createTask()                POST /api/tasks/
      → DRF serializer validates  → Task.objects.create()
        → 201 Created + JSON      → setTasks([saved, ...tasks])
          → TaskList re-renders    → the new row appears
```

We build it **bottom-up**: model → serializer → view → route, then service → state → components.

The REST contract

Method	URL	What it does
`GET`	`/api/tasks/`	list every task, newest first
`POST`	`/api/tasks/`	create a task, returns 201 + the new row
`GET`	`/api/tasks/5/`	read one task
`PUT`	`/api/tasks/5/`	replace a task (all fields required)
`PATCH`	`/api/tasks/5/`	update some fields — what we use for toggling
`DELETE`	`/api/tasks/5/`	delete a task, returns 204 No Content

> The **trailing slash is required**. DRF's `DefaultRouter` builds these URLs; `/api/tasks` without the slash gets redirected and a POST body can be lost.

Backend Setup

Virtualenv first

```
mkdir task-manager && cd task-manager
mkdir backend && cd backend

python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

pip install -r requirements.txt
```

- A virtualenv keeps this project's packages out of your system Python.
- Activate it in **every new terminal** — the `(.venv)` prefix is your proof.
- Never commit `.venv/`.

requirements.txt

```
Django>=5.2,<6.0
djangorestframework>=3.16,<4.0
django-cors-headers>=4.7,<5.0
python-dotenv>=1.1,<2.0
PyMySQL>=1.1,<2.0
```

- `djangorestframework` — serializers, viewsets, the browsable API.
- `django-cors-headers` — lets `localhost:5173` call `127.0.0.1:8000`.
- `python-dotenv` — reads `.env` so secrets stay out of the code.
- `PyMySQL` — pure-Python MySQL driver, no C compiler needed.

Create the project and the app

```
django-admin startproject task_manager .   # the "." matters: no extra nesting
python manage.py startapp tasks
```

> Without the trailing `.` you get `backend/task_manager/task_manager/`. **Project** = configuration. **App** = one feature. Here: project `task_manager`, app `tasks`.

Backend folder structure

```
backend/
  .venv/                  # not committed
  .env                    # real secrets   - never commit
  .env.example            # safe template  - do commit
  manage.py
  requirements.txt
  task_manager/           # the project
    settings.py           # env, apps, database, CORS
    urls.py               # root routes
    wsgi.py
  tasks/                  # the app
    models.py             # the table
    serializers.py        # JSON in / JSON out
    views.py              # the endpoints
    urls.py               # the routes
    admin.py
    migrations/
```

Secrets in .env

```
# .env
SECRET_KEY=change-this-secret-key
DEBUG=True
ALLOWED_HOSTS=127.0.0.1,localhost

MYSQL_DATABASE=task_manager_db
MYSQL_USER=root
MYSQL_PASSWORD=
MYSQL_HOST=127.0.0.1
MYSQL_PORT=3306

CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

> Copy it to `.env.example` with the values blanked out and commit **that**, so the next developer knows which keys to set.

Ignore it in git

```
# backend/.gitignore
.venv/
__pycache__/
*.pyc
.env
db.sqlite3
```

⚠️ Unlike a frontend key, this one really is secret — it never reaches the browser. Leaking `SECRET_KEY` lets anyone forge Django session cookies. **Never commit `.env`.**

settings.py — environment

```
from pathlib import Path
import os
from dotenv import load_dotenv
import pymysql

pymysql.install_as_MySQLdb()      # make Django's mysql backend use PyMySQL

BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")    # read .env into os.environ

SECRET_KEY = os.getenv("SECRET_KEY", "dev-only-secret-key")
DEBUG = os.getenv("DEBUG", "True") == "True"
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "127.0.0.1,localhost").split(",")
```

`os.getenv` always returns a **string**, and the string `"False"` is truthy — that is why we compare with `== "True"`.

settings.py — apps and middleware

```
INSTALLED_APPS = [
    # ... the six django.contrib defaults ...
    "corsheaders",        # third-party
    "rest_framework",
    "tasks",              # our app
]

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",          # must come first
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    # ... the rest unchanged ...
]
```

Middleware order is real. `CorsMiddleware` must sit **above** `CommonMiddleware`, or the CORS headers never get attached to the response.

settings.py — MySQL

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.mysql",
        "NAME": os.getenv("MYSQL_DATABASE", "task_manager_db"),
        "USER": os.getenv("MYSQL_USER", "root"),
        "PASSWORD": os.getenv("MYSQL_PASSWORD", ""),
        "HOST": os.getenv("MYSQL_HOST", "127.0.0.1"),
        "PORT": os.getenv("MYSQL_PORT", "3306"),
        "OPTIONS": {
            "charset": "utf8mb4",
        },
    }
}
```

`utf8mb4` is real 4-byte Unicode — emoji in a task title work. MySQL's old `utf8` is only 3 bytes and will reject them.

Create the database yourself

```
-- mysql -u root -p
CREATE DATABASE task_manager_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

Django creates **tables**, not databases. Make the empty database once by hand; `migrate` fills it. Wrong name or password here shows up later as `django.db.utils.OperationalError`.

settings.py — CORS and DRF

```
CORS_ALLOWED_ORIGINS = [
    origin.strip()
    for origin in os.getenv(
        "CORS_ALLOWED_ORIGINS",
        "http://localhost:5173,http://127.0.0.1:5173",
    ).split(",")
    if origin.strip()
]

REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.AllowAny",
    ],
}
```

The browser blocks `:5173 → :8000` unless Django replies with `Access-Control-Allow-Origin`. List **both** `localhost` and `127.0.0.1` — they are different origins. `AllowAny` is fine for class; tighten it before production.

The API

tasks/models.py

```python
from django.db import models

class Task(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)             # optional
    is_completed = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)   # stamped once
    updated_at = models.DateTimeField(auto_now=True)       # stamped every save

    class Meta:
        ordering = ["-created_at"]      # newest first, everywhere

    def __str__(self):
        return self.title
```

`blank=True` means "may be empty when validating". `auto_now_add` fires only on insert, `auto_now` on every save — both are read-only to clients.

Migrate

```
python manage.py makemigrations   # writes tasks/migrations/0001_initial.py
python manage.py migrate          # runs the CREATE TABLE against MySQL
```

- `makemigrations` = write the plan. `migrate` = apply it.
- The table is named `tasks_task` — `<app>_<model>`, lowercased.
- **Commit migration files.** They are the schema's history.
- Change `models.py` later? Run both commands again.

tasks/serializers.py

```python
from rest_framework import serializers

from .models import Task

class TaskSerializer(serializers.ModelSerializer):
    class Meta:
        model = Task
        fields = [
            "id", "title", "description",
            "is_completed", "created_at", "updated_at",
        ]
        read_only_fields = ["id", "created_at", "updated_at"]
```

A serializer is a **translator both ways**: model → JSON on the way out, JSON → validated data on the way in. `read_only_fields` means a client that POSTs an `id` is politely ignored.

tasks/views.py

```python
from rest_framework import viewsets

from .models import Task
from .serializers import TaskSerializer

class TaskViewSet(viewsets.ModelViewSet):
    queryset = Task.objects.all()          # what rows this endpoint sees
    serializer_class = TaskSerializer      # how to convert them
```

`ModelViewSet` is list + create + retrieve + update + partial_update + destroy, already written. Two lines of configuration buy you six endpoints.

What the viewset generates

HTTP	URL	ViewSet action
GET	/api/tasks/	list()
POST	/api/tasks/	create()
GET	/api/tasks/5/	retrieve()
PUT	/api/tasks/5/	update()
PATCH	/api/tasks/5/	partial_update()
DELETE	/api/tasks/5/	destroy()

> Need custom behaviour later — filtering by user, an extra action — you override **one method** and the other five keep working.

tasks/urls.py — the router

```python
from django.urls import include, path
from rest_framework.routers import DefaultRouter

from .views import TaskViewSet

router = DefaultRouter()
router.register("tasks", TaskViewSet, basename="task")

urlpatterns = [
    path("", include(router.urls)),
]
```

The router turns one viewset into the whole URL table. `basename` names the reverse URLs, e.g. `task-list`, `task-detail`.

task_manager/urls.py

```python
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("tasks.urls")),   # /api/  +  tasks/  =  /api/tasks/
]
```

Prefixes stack: the project mounts the app under `/api/`, the app's router adds `tasks/`. Keep every API route behind one prefix.

Admin, for a free UI

```
# tasks/admin.py
from django.contrib import admin

from .models import Task

admin.site.register(Task)
```

```
python manage.py createsuperuser
python manage.py runserver       # then open /admin/
```

Test the API before touching React

```
python manage.py runserver                    # http://127.0.0.1:8000

curl http://127.0.0.1:8000/api/tasks/

curl -X POST http://127.0.0.1:8000/api/tasks/ \
  -H "Content-Type: application/json" \
  -d '{"title": "Write the API", "description": "DRF viewset"}'
```

> Or just open `http://127.0.0.1:8000/api/tasks/` in the browser — DRF renders a **browsable API** with a form. If the API is broken here, no amount of React will fix it.

Frontend Setup

Scaffold with Vite

```
cd ..                          # back up to task-manager/
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev                    # http://localhost:5173
```

- Vite gives an instant dev server and fast HMR.
- `pnpm` or `yarn` work the same — swap the command.
- Leave Django running in its own terminal.

Add Tailwind and icons

```
npm install tailwindcss @tailwindcss/vite
npm install lucide-react
```

- **Tailwind v4** is a Vite plugin — no `tailwind.config.js`, no PostCSS file.
- **lucide-react** ships icons as React components: `<Check size={18} />`.
- Utility classes mean no CSS files to name, nest and maintain.

Wire up Tailwind

```
// vite.config.js
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

```
/* src/index.css */
@import "tailwindcss";
```

Frontend folder structure

```
frontend/
  index.html
  vite.config.js
  src/
    main.jsx              # mounts <App /> into #root
    index.css             # the Tailwind import
    App.jsx               # all state + all handlers
    services/
      taskApi.js          # every fetch call lives here
    components/
      TaskForm.jsx        # controlled create form
      TaskList.jsx        # the list + the empty state
      TaskItem.jsx        # one row + its two buttons
```

Three layers: **services/** talks HTTP, **App.jsx** owns state, **components/** only render and call props.

The Service Layer

services/taskApi.js — read and create

```
const API_BASE_URL = "http://127.0.0.1:8000/api";

export async function getAllTasks() {
  const response = await fetch(`${API_BASE_URL}/tasks/`);
  if (!response.ok) {
    throw new Error("Failed to fetch tasks");   // fetch does NOT throw on 404/500
  }
  return response.json();
}

export async function createTask(task) {
  const response = await fetch(`${API_BASE_URL}/tasks/`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(task),
  });
  if (!response.ok) throw new Error("Failed to create task");
  return response.json();
}
```

services/taskApi.js — update and delete

```
export async function patchTask(id, task) {
  const response = await fetch(`${API_BASE_URL}/tasks/${id}/`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(task),
  });
  if (!response.ok) throw new Error("Failed to update task");
  return response.json();
}

export async function deleteTask(id) {
  const response = await fetch(`${API_BASE_URL}/tasks/${id}/`, {
    method: "DELETE",
  });
  if (!response.ok) throw new Error("Failed to delete task");
  // 204 No Content — there is no body to parse
}
```

Three rules for this file

- `fetch` **only rejects on network failure.** A 400 or a 500 still resolves — you must check `response.ok` yourself.
- **Throw here, catch in the component.** Every function either returns data or throws; no `{ ok, data, error }` objects to unpack.
- **PATCH, not PUT, for the toggle.** `PUT` replaces the row and demands every field; `PATCH` sends `{ is_completed: true }` and nothing else.
- The base URL sits in one `const` — one line to change for deployment.

State & Components

App.jsx — state design

```
export default function App() {
  const [tasks, setTasks] = useState([]);            // the server's rows
  const [isLoading, setIsLoading] = useState(true);  // first load only
  const [isSaving, setIsSaving] = useState(false);   // disables the Add button
  const [error, setError] = useState("");            // one message for all calls

  // Derived, not stored: recomputed on every render, never stale.
  const completedCount = tasks.filter((task) => task.is_completed).length;
}
```

Derive, don't duplicate. `completedCount` is not state — a second `useState` for it would drift out of sync the moment a task is deleted.

App.jsx — load once on mount

```
useEffect(() => {
  loadTasks();
}, []);                          // [] = run once, after the first render

async function loadTasks() {
  try {
    setError("");
    const data = await getAllTasks();
    setTasks(data);
  } catch (apiError) {
    setError(apiError.message);
  } finally {
    setIsLoading(false);        // runs on success AND on failure
  }
}
```

Drop the `[]` and the effect runs after **every** render — each fetch sets state, which triggers a render, which fetches again. Infinite loop.

App.jsx — add a task

```
async function handleAddTask(newTask) {
  try {
    setIsSaving(true);
    setError("");
    const savedTask = await createTask(newTask);
    setTasks((currentTasks) => [savedTask, ...currentTasks]);
  } catch (apiError) {
    setError(apiError.message);
  } finally {
    setIsSaving(false);
  }
}
```

Prepend the **server's** row, not the object you sent — only the response has `id`, `created_at` and `updated_at`. Newest first matches the model's `ordering`, so a refresh shows the same order.

App.jsx — toggle and delete

```
async function handleToggleTask(id, changes) {
  const updatedTask = await patchTask(id, changes);      // { is_completed: true }
  setTasks((currentTasks) =>
    currentTasks.map((task) => (task.id === id ? updatedTask : task)),
  );
}

async function handleDeleteTask(id) {
  await deleteTask(id);
  setTasks((currentTasks) => currentTasks.filter((task) => task.id !== id));
}
```

Shown without the `try / catch (apiError) { setError(...) }` wrapper for space — in the real file both have it. `map` **to replace,** `filter` **to remove** — never mutate the array you already have.

App.jsx — the render branches

```
<TaskForm onAddTask={handleAddTask} isSaving={isSaving} />

{error ? (
  <p className="rounded-md border border-rose-200 bg-rose-50 ...">{error}</p>
) : null}

{isLoading ? (
  <p className="...">Loading tasks...</p>
) : (
  <TaskList
    tasks={tasks}
    onToggleTask={handleToggleTask}
    onDeleteTask={handleDeleteTask}
  />
)}
```

> Use `? :` or `? ... : null` — **not** `error && <p>`. With `&&`, a value like `0` or `""` renders itself instead of nothing.

TaskForm.jsx — a controlled form

```
export default function TaskForm({ onAddTask, isSaving }) {
  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");

  function handleSubmit(event) {
    event.preventDefault();          // stop the browser reloading the page
    if (!title.trim()) return;       // ignore empty titles

    onAddTask({
      title: title.trim(),
      description: description.trim(),
      is_completed: false,
    });

    setTitle("");                    // clear the inputs
    setDescription("");
  }
}
```

TaskForm.jsx — the inputs

```
<input
  id="title"
  value={title}                                    // React owns the value
  onChange={(event) => setTitle(event.target.value)}
  placeholder="Example: Finish Django API"
  className="w-full rounded-md border border-slate-300 px-3 py-2 ..."
/>

<button type="submit" disabled={isSaving} className="...">
  <Plus size={18} />
  {isSaving ? "Adding..." : "Add Task"}
</button>
```

Controlled = `value` + `onChange` together. Give a `value` with no `onChange` and the field is frozen; React warns in the console.

TaskList.jsx — list and empty state

```
export default function TaskList({ tasks, onToggleTask, onDeleteTask }) {
  if (tasks.length === 0) {
    return (
      <div className="rounded-lg border border-dashed ...">
        No tasks yet. Add your first task above.
      </div>
    );
  }

  return (
    <ul className="grid gap-3">
      {tasks.map((task) => (
        <TaskItem key={task.id} task={task}
                  onToggleTask={onToggleTask} onDeleteTask={onDeleteTask} />
      ))}
    </ul>
  );
}
```

`key={task.id}` — the database id, never the array index. With an index key, deleting the first row makes React reuse the wrong DOM node.

TaskItem.jsx — one row

```jsx
<h2 className={task.is_completed ? "text-slate-500 line-through" : "text-slate-900"}>
  {task.title}
</h2>

{task.description ? <p className="...">{task.description}</p> : null}
<p className="mt-3 text-xs text-slate-400">
  Created {new Date(task.created_at).toLocaleString()}
</p>

<button onClick={() => onToggleTask(task.id, { is_completed: !task.is_completed })}>
  {task.is_completed ? <Undo2 size={18} /> : <Check size={18} />}
</button>

<button onClick={() => onDeleteTask(task.id)}><Trash2 size={18} /></button>
```

`TaskItem` holds **no state**. It takes data down as props and sends events up through callbacks — that is why the counter in the header is always right.

Run & Debug

Two terminals, two servers

```
# terminal 1 — backend
cd backend
source .venv/bin/activate
python manage.py runserver          # http://127.0.0.1:8000

# terminal 2 — frontend
cd frontend
npm run dev                         # http://localhost:5173
```

> Open the frontend URL. If the list stays empty, open **DevTools → Network** and look at the `/api/tasks/` request before you touch any React code.

The five errors you will hit

- **CORS blocked** — `corsheaders` missing from `INSTALLED_APPS`, `CorsMiddleware` not first, or your port not in `CORS_ALLOWED_ORIGINS`.
- **POST silently redirects** — you dropped the trailing slash: `/api/tasks/`, not `/api/tasks`.
- **400 Bad Request** — title empty or over 200 characters. `await response.json()` shows exactly which field failed.
- **OperationalError** — MySQL isn't running, the password is wrong, or you never created `task_manager_db`.
- **Nothing renders** — `key` warnings, or state replaced with `undefined` because a service function returned nothing.

Where to go next

- Swap the fetch plumbing for **TanStack React Query** — caching, retries and `isLoading` for free, and `useEffect` disappears.
- Add **edit-in-place** with `PUT`, and filters: all / active / done.
- Add **users + JWT** and scope `queryset` to `request.user`.
- Add **pagination** and `django-filter` once the list gets long.
- Deploy: Django on Railway or Render, React on Vercel or Netlify, with the API base URL moved into `VITE_API_URL`.

🧪 Build it (live, 25 min)

1. Virtualenv, install, `startproject` + `startapp`, fill `.env`.
2. Point `settings.py` at MySQL; add `corsheaders`, `rest_framework`, `tasks`.
3. Write `models.py`, then `makemigrations` + `migrate`.
4. Serializer → `ModelViewSet` → router → `/api/tasks/`. Test with curl.
5. Scaffold Vite, add Tailwind, create the three folders.
6. Write `services/taskApi.js` — all four calls.
7. Wire `App.jsx`: state, `useEffect`, three handlers, render branches.
8. Build `TaskForm` → `TaskList` → `TaskItem`. Run both servers.

Recap

Project = settings and root URLs. **App** = one feature, one folder.

Model → migration → serializer → viewset → router. Six endpoints, ten lines.

Secrets live in `.env`, are read with `python-dotenv`, and are gitignored.

CORS is a **browser** rule — `CorsMiddleware` first, both origins listed.

Every `fetch` lives in `services/`; check `response.ok` and throw.

`App.jsx` owns state, components only render. Derive counts, never store them.

Trailing slashes, `PATCH` over `PUT`, `key={task.id}` — the three quiet bugs.

Let's connect

Questions after class? Reach out any time.

🌐 Portfolio: **tausif.bd**

💼 LinkedIn: **linkedin.com/in/tausif1337**

💻 GitHub: **github.com/tausif1337**

💬 WhatsApp: **wa.me/8801748181448**

Keep building. See you next session.